- Phase 2: Property Retrieval — Supabase Query Integration
  - 1. Overview
  - 2. Available Data at SHOW_RESULTS
  - 3. Database Schema (Read-Only)
    - properties Table
  - 4. Filtering Logic
    - Deterministic Filter Application
    - Filter Application Order (for determinism)
  - 5. Result Limit & Sorting
    - Row Limit
    - Sorting
  - 6. Response Structure
    - New Response Format
    - Example Response (SHOW_RESULTS step)
  - 7. Fallback Behavior (Missing Filters)
    - Scenario 1: No Location Provided
    - Scenario 2: Budget Missing (null budget_min/max)
    - Scenario 3: No Matches Found
    - Scenario 4: Partial Matches
  - 8. Service Architecture
    - New File: lib/propertyService.ts
    - Integration with chatService
  - 9. Logging Requirements
    - propertyService.ts Logging
    - chatService.ts Logging (SHOW_RESULTS case)
  - 10. Type Safety & TypeScript
    - New Types in types/chat.ts
    - Query Type Safety
  - 11. Anti-Patterns & Constraints
    - ❌ DO NOT:
    - ✅ DO:
  - 12. Validation Checklist
  - 13. Specification Summary
  - Next Steps (Upon Approval)

Phase 2: Property Retrieval — Supabase Query Integration

Status: Specification (Awaiting Approval)
Date: April 11, 2026
Objective: Replace placeholder property lookup logic in SHOW_RESULTS with real Supabase queries returning structured property data.

---

1. Overview

Phase 2 extends the Phase 1 deterministic chat flow by:

- Replacing `getResultSummary()` placeholder with structured property queries
- Returning full property objects (not just titles) with id, title, location, price, bhk
- Implementing conditional, deterministic SQL-based filtering
- Handling graceful fallback when filters are incomplete
- Supporting up to 5 results per request

Key Constraint: All filtering must be deterministic SQL-based. No AI, no in-memory filtering, no post-query processing.

---

2. Available Data at SHOW_RESULTS

When the flow reaches `SHOW_RESULTS` step, the conversation's `collected_data` contains:

Field	Type	Status
`intent`	'buy' | 'rent' | 'explore'	✅ Always present
`location`	string	✅ Always present (normalized: trim + lowercase)
`budget_min`	number | null	⚠️ May be null (no budget provided)

Field	Type	Status
`budget_max`	number | null	⚠️ May be null (no budget provided)
`property_type`	'apartment' | 'villa' | 'plot' | 'commercial'	✅ Always present
`bhk`	string	✅ Always present (user input, e.g., "2BHK")
`timeline`	'urgent' | 'soon' | 'flexible'	✅ Always present

Note: `intent`, `property_type`, and `timeline` are guaranteed from Phase 1 validation.

---

3. Database Schema (Read-Only)

`properties` Table

Column	Type	Nullable	Notes
`id`	UUID	No	Primary key
`title`	TEXT	No	Listing title
`location`	TEXT	No	Area/locality in Visakhapatnam
`price`	INT	No	Price in rupees
`type`	TEXT	Yes	Property category (apartment/villa/plot/commercial)
`bhk`	TEXT	Yes	BHK configuration (e.g., "2BHK", "1BHK")
`is_available`	BOOLEAN	No	Availability flag (default true)
`status`	TEXT	Yes	Listing status
`description`	TEXT	Yes	Full description
`agent_id`	UUID	Yes	Assigned agent

Column	Type	Nullable	Notes
`created_at`	TIMESTAMP	No	Creation datetime

Availability Constraint: Always query with `is_available = true`.

---

4. Filtering Logic

Deterministic Filter Application

Filters are applied conditionally based on collected data availability:

```
ALWAYS APPLY:
  ✅ is_available = true

CONDITIONALLY APPLY:
  📍 IF location is provided:
     → Apply ILIKE filter: location ~* `%${location}%` (case-insensitive
substring match)
     → Rationale: Location may contain partial names (e.g., "ram" matches
"Rambagh")

  💰 IF budget_min is a number:
     → Apply GTE filter: price >= budget_min

  💰 IF budget_max is a number:
     → Apply LTE filter: price <= budget_max

  🏠 IF property_type is provided:
     → Apply EQ filter: type = property_type
     → Note: User input "apartment" maps directly to properties.type column

  🛏️ IF bhk is provided:
     → Apply EQ filter: bhk = bhk
     → Rationale: Exact match on user input (e.g., "2BHK" must match
exactly)
```

Filter Application Order (for determinism)

1. `is_available = true`
2. `location` (ILIKE if provided)
3. `type` (EQ if property_type provided)

4. `price` (GTE if budget_min, LTE if budget_max)
5. `bhk` (EQ if provided)

This order ensures consistent query plan generation.

---

5. Result Limit & Sorting

Row Limit

- **Maximum:** 5 properties per query
- **Rationale:** Prevents overwhelming user with options; matches Phase 1 `getResultSummary` behavior (top 3 titles)

Sorting

- **Primary:** `created_at DESC` (newest first)
- **Rationale:** Provides deterministic ordering independent of any "relevance" or AI scoring. Newest listings are typically higher engagement signals.

---

6. Response Structure

New Response Format

The API response must be updated to include property details:

```
interface Property {
  id: string        // UUID
  title: string     // Listing title
  location: string  // Area/locality
  price: number     // Price in rupees
  bhk?: string      // BHK configuration (may be null in DB)
}

interface ChatApiResponse {
  // ... existing fields ...
```

```
  // NEW FIELD (included when step = SHOW_RESULTS)
  properties?: Property[]
}
```

Example Response (SHOW_RESULTS step)

```
{
  "success": true,
  "sessionId": "uuid...",
  "conversationId": "uuid...",
  "step": "SHOW_RESULTS",
  "response": "Great! I found 5 properties matching your criteria.",
  "properties": [
    {
      "id": "uuid-1",
      "title": "Beautiful 2BHK in Visakhapatnam",
      "location": "Rambagh",
      "price": 3500000,
      "bhk": "2BHK"
    },
    {
      "id": "uuid-2",
      "title": "Premium Villa near Beach",
      "location": "Vizag Beach Road",
      "price": 4200000,
      "bhk": "3BHK"
    },
    // ... up to 5 total ...
  ],
  "collectedData": { /* ... */ }
}
```

---

7. Fallback Behavior (Missing Filters)

Scenario 1: No Location Provided

Condition: `location` is empty or whitespace

Action: Skip location filter; return latest 5 properties matching other filters

Example Query:

```
SELECT * FROM properties
WHERE is_available = true
```

```
  AND type = 'apartment'
  AND price BETWEEN 2000000 AND 5000000
ORDER BY created_at DESC
LIMIT 5
```

Scenario 2: Budget Missing (null budget_min/max)

Condition: Both `budget_min` and `budget_max` are null

Action: Skip price filters entirely

Example Query:

```
SELECT * FROM properties
WHERE is_available = true
  AND location ILIKE '%visakhapatnam%'
  AND type = 'apartment'
ORDER BY created_at DESC
LIMIT 5
```

Scenario 3: No Matches Found

Condition: Query returns 0 rows

Action: Return empty properties array with fallback response message

Response:

```
{
  "success": true,
  "step": "SHOW_RESULTS",
  "response": "No exact matches found, but the agent can help you further.",
  "properties": [],
  "collectedData": { /* ... */ }
}
```

Rationale: User flow continues to CAPTURE_NAME step for escalation/follow-up.

Scenario 4: Partial Matches

Condition: Query returns fewer than 5 results

Action: Return all available results (no padding, no defaults)

Response:

```
{
  "success": true,
  "step": "SHOW_RESULTS",
  "response": "I found 2 properties matching your criteria.",
  "properties": [ /* 2 items */ ],
  "collectedData": { /* ... */ }
}
```

---

8. Service Architecture

New File: `lib/propertyService.ts`

Purpose: Encapsulate all deterministic property query logic.

Exports:

```
export interface PropertyFilter {
  location?: string        // Optional location filter
  budget_min?: number | null
  budget_max?: number | null
  property_type?: string   // Mapped from collected_data.property_type
  bhk?: string            // Exact match on bhk
}

export interface PropertyResult {
  id: string
  title: string
  location: string
  price: number
  bhk?: string
}

/**
 * Deterministic query to fetch matching properties.
 *
 * Rules:
 * - Always filters: is_available = true
 * - Applies optional filters only if provided
 * - Sorts by created_at DESC (newest first)
 * - Limits to 5 results
 * - Returns empty array if no matches
```

```
 *
 * @param filters - Conditional filter criteria
 * @returns Promise<PropertyResult[]> - Up to 5 matching properties
 */
export async function getMatchingProperties(filters: PropertyFilter):
Promise<PropertyResult[]>
```

Integration with chatService

In `lib/chatService.ts`, the `processConversationTurn()` function must be updated:

Current Behavior (SHOW_RESULTS case):

```
case ChatStep.SHOW_RESULTS: {
  const summary = await getResultSummary(conversation)
  const titles = summary.topTitles.length > 0 ? summary.topTitles.join(', ')
: 'no exact title matches yet'
  response = `Great! I found ${summary.count} properties matching your
criteria. Top matches: ${titles}.`
  conversation.step = getNextStep(conversation.step)
  break
}
```

New Behavior (Phase 2):

```
case ChatStep.SHOW_RESULTS: {
  const filters: PropertyFilter = {
    location: conversation.collected_data.location,
    budget_min: conversation.collected_data.budget_min,
    budget_max: conversation.collected_data.budget_max,
    property_type: conversation.collected_data.property_type,
    bhk: conversation.collected_data.bhk
  }

  const properties = await getMatchingProperties(filters)
  const count = properties.length
  const message = count === 0
    ? "No exact matches found, but the agent can help you further."
    : `Great! I found ${count} properties matching your criteria.`

  response = message

  // Store properties in conversation for response (not persisted to DB)
  conversation.properties = properties

  conversation.step = getNextStep(conversation.step)
```

```
    break
  }
```

Action Items:

1. Remove `getResultSummary()` function (no longer needed)
2. Create `propertyService.ts` with `getMatchingProperties()`
3. Import and call `getMatchingProperties()` in SHOW_RESULTS case
4. Store properties in temporary conversation object (not in DB)

---

9. Logging Requirements

propertyService.ts Logging

```
console.log('[property-service] query built', {
  filters: {
    location: filters.location,
    budget_min: filters.budget_min,
    budget_max: filters.budget_max,
    property_type: filters.property_type,
    bhk: filters.bhk
  },
  expectedCount: 'TBD after query'
})

console.log('[property-service] query result', {
  count: properties.length,
  maxResults: 5,
  filters_applied: {
    location: !!filters.location,
    budget_min: typeof filters.budget_min === 'number',
    budget_max: typeof filters.budget_max === 'number',
    property_type: !!filters.property_type,
    bhk: !!filters.bhk
  }
})
```

chatService.ts Logging (SHOW_RESULTS case)

```
console.log('[chat] showing results', {
  sessionId: conversation.session_id,
  count: properties.length,
  collected_data: conversation.collected_data
})
```

---

10. Type Safety & TypeScript

New Types in `types/chat.ts`

Add to the module:

```
// Phase 2 types
export interface PropertyResult {
  id: string
  title: string
  location: string
  price: number
  bhk?: string
}

// Extended ChatApiResponse (add optional field)
export interface ChatApiResponse {
  // ... existing fields ...
  properties?: PropertyResult[]  // Included when step = SHOW_RESULTS
}

// Extended ConversationRecord (temporary field, not persisted)
export interface ConversationRecord {
  // ... existing fields ...
  properties?: PropertyResult[]  // Temporary, for current response only
}
```

Query Type Safety

Use generated Supabase types from `types/supabase.ts`:

```
import { Tables } from '@/types/supabase'

type PropertiesRow = Tables<'properties'>
```

```
// In getMatchingProperties:
const { data, error } = await supabase
  .from('properties')
  .select('id,title,location,price,bhk')
  .eq('is_available', true)
  // ... filters ...

// TypeScript ensures selected columns match PropertiesRow fields
```

---

11. Anti-Patterns & Constraints

❌ DO NOT:

- Use AI/LLM for filtering or ranking
- Fetch all properties and filter in JavaScript
- Apply price filters if both budget_min and budget_max are null
- Return full property objects with description, agent_id, or other metadata
- Write query results to database
- Assume bhk format (accept user input as-is)
- Use vector search or embeddings
- Modify database schema
- Change the existing API contract for other steps

✅ DO:

- Filter by `is_available = true` on every query
- Return exactly: id, title, location, price, bhk
- Limit to 5 results max
- Sort by `created_at DESC` for determinism
- Handle null budget gracefully
- Log filter application for debugging
- Use Supabase typed client for type safety
- Test with missing/partial filters

---

12. Validation Checklist

Before approving implementation:

- ☐ Properties table schema reviewed in `types/supabase.ts`
- ☐ Filter logic aligns with collected_data structure
- ☐ Response format extends API contract without breaking changes
- ☐ Fallback behavior covers all missing filter scenarios
- ☐ Deterministic SQL sorting (created_at DESC) is justified
- ☐ Result limit (5) is acceptable for UX
- ☐ TypeScript types are defined for PropertyResult
- ☐ Logging strategy captures filter application
- ☐ No AI/LLM usage in filtering logic
- ☐ Integration point (SHOW_RESULTS case) identified

---

13. Specification Summary

Aspect	Decision
Data Source	Supabase `properties` table (read-only)
Filter Type	Deterministic SQL-based (conditional)
Availability	Always filter `is_available = true`
Result Limit	5 properties max
Sort Order	`created_at DESC` (newest first)
Location Match	Case-insensitive substring (ILIKE)
Price Match	Range (GTE budget_min, LTE budget_max)
Type Match	Exact match on property_type
BHK Match	Exact match on user input
Empty Result	Return [] with fallback message
Partial Result	Return all available (< 5)
Response Field	Add optional `properties: PropertyResult[]`
Logging	Filter criteria + result count
Type Safety	Full TypeScript with generated types

Aspect	Decision
Schema Changes	None (read-only)

Next Steps (Upon Approval)

1. **Implementation** → Create `lib/propertyService.ts`
2. **Update** → Integrate `getMatchingProperties()` into chatService.ts
3. **Types** → Update `types/chat.ts` with PropertyResult interface
4. **Response** → Update API response to include properties field
5. **Testing** → Validate with various filter combinations
6. **Validation** → Confirm deterministic behavior (same input → same output)

Specification prepared for Senior Backend Engineer review.

Awaiting approval before implementation begins.